singlecomm

ENGINEERING

# SingleConsole

A testable agent console — and the first time we can see what our fleet is actually doing, including the consoles that have stopped talking to us.

> The classic console speaks 116 AJAX verbs and 36 signalling frame types, in PHP 7.4 and jQuery, with no build step and no seam to test at. You cannot refactor your way out of that. You can only **grow something beside it** — and then let it strangle the old tree.

**41**
workspace packages behind one headless kernel

**3,533**
unit tests; coverage is a gate, not a report

**4**
channels proven end to end: inbound, outbound, omni, meet

**65 kB**
gzipped widget bundle, embeddable anywhere

## React renders. The kernel decides.

All state lives in a kernel with **no DOM dependency**. Shells are interchangeable, which is what makes four deployments one product rather than four forks — and it is why the expensive part of a desktop or native client is already written.

**Built in** SHIPPED
A module inside the platform — "Agent Desktop". Same page, same session, same permissions.

**Widget** SHIPPED
A bundle that drops into third-party products — CRMs, ticketing, whatever the client already lives in.

**Desktop** IN PROGRESS
An Electron shell running the **same codebase** as the browser build — not a parallel implementation.

**Native** PLANNED
Rust / Qt, once the web version has stabilised, and kept in step with it from then on.

## Control Tower — and the state the old console could never see

Every console beacons roughly every twenty seconds, and the Tower reads four **liveness** states from that — whether we are still hearing from a machine at all. This is not agent status, which is about what the person is doing.

| | |
|---|---|
| **live** | beaconing normally |
| **silent** | **expected, and not heard from.** The one that matters — a console whose network has gone does not report the outage, it **stops reporting**. The board gets quieter while people cannot work. |
| **ended** | signed out cleanly, with a reason — a final beacon on the way out |
| **vanished** | stopped mid-shift with no goodbye |

### The number that reframes everything

A census of **1,387 real production consoles** found that **93% are not the focused window and 69% are not even visible.** Browsers throttle a hidden tab's timers to roughly one a minute — so a console behind a spreadsheet beacons late and looks exactly like one in trouble. On a 900-console fleet that is **forty of eighty-six quiet consoles that are completely fine**. Saying so is forty phone calls not made.

## A flight recorder that cannot leak

Each session carries a sealed, size-bounded, hash-chained event ring, plus **a manifest declaring what is absent** — evicted, decimated or stripped. Redaction is by construction rather than by filter: a registry declares, per field, whether it is allowed through, reduced to a shape, or dropped and counted. Unregistered event types cannot be recorded at all.

Never captured, by design: message bodies, caller name or number, screen contents, audio, script field values, raw device identifiers.

## Rescue — evidence off a machine that cannot reach us

Browser alive, network gone. The console walks a ladder — `healthy → degraded → impacted → isolated → dark` — and only `dark`, held twenty seconds, raises a **flare**: a **QR code** carrying a sealed capsule with the workstation, the session, the time and the console's state. Someone photographs the screen. The code points at **the agent's own cluster**, deliberately not at whatever origin served the page, so the scan lands beside the session it belongs to. A **spoken short code** comes first in the reading order, because the person who needs this may be at a dark monitor with no camera.

## The AI play: Watch and Review

| | |
|---|---|
| **Watch** | **What we can prove.** Deterministic rules, live, in-process. A rule that is **certain** can raise a finding immediately, tell the agent mid-call, and drive a repair with no human in the loop. Certainty is a distinct kind of answer, not a high score. |
| **Review** | **What we suspect.** An overnight pass over redacted evidence. Where no deterministic rule matched, a model proposes — labelled **inferred**, with its reasoning, how sure it is, and **what would prove it wrong**. Sometimes the output is a new rule, together with the backtest showing it would have caught the thing. |

**We don't put models in the loop. We use models to find the rules — then run the rules without the model.**

### The four boundaries that make that true

- **It suggests, people decide, the engine acts.** Whatever runs is something a human already reviewed, so a bad suggestion costs a minute rather than an incident.
- **Pick from a list before writing prose.** "Which of these known problems is this?" — with "none of them" always allowed. Free text is a fallback, labelled unverified wherever it appears, and never an action button.
- **Evidence, not raw dumps.** Summarised measurements and a redacted bundle. Scoped to one tenant, and switchable off per tenant without disabling any deterministic layer.
- **Every suggestion is scored afterwards.** Shown, accepted, run — and then, did it actually work? One that keeps working becomes a rule we ship. **The system improves without a model anywhere near the runtime**, because the learning lands as reviewed, versioned, switch-off-able data.

### sctower — the same tower, for people and for agents

```
sctower auth login   # OAuth 2.1 + PKCE, loopback
sctower live · digest · session <id> --tab timeline
sctower probe <id> · rescue <code> · ui
```

One static binary, shipped with **its own skill** so an agent can pick it up without being taught. `sctower watch --since 1h && echo clean` — exit **1** means **answered, and the answer is bad**; exit **4** means unreachable. An agent that can tell those apart can triage; one that cannot will hallucinate the difference.

## Where it stands

| | |
|---|---|
| **Shipped** | The kernel, four channels, the built-in module and the embeddable widget · the flight recorder · Control Tower's live and retrospective halves · self-repair and silence detection · the CLI and its terminal UI |
| **In flight** | Rescue in every shipped shell · the desktop build · Review running against a real model · post-call detection from the call records we already write |
| **Designed** | In-call detection from the live media plane · a cross-cluster view for operations · a native client |